



FitPlanner - Documento di Commissione e Specifiche Tecniche (v3.0)

Progetto: FitPlanner - Piattaforma Web Full-Stack per Personal Trainer

Obiettivo: Sviluppare, mettere in sicurezza e pubblicare una web app per la gestione di schede di allenamento.

Dominio target: www.Lorenzoanzivinocoach.it (Hosting Ionos/VPS)

1. Panoramica e Obiettivi

Il software deve permettere la gestione completa del ciclo di lavoro tra Personal Trainer e Cliente.

- **Il Trainer** deve poter creare schede complesse, organizzarle in cartelle, e monitorare i progressi.
- **Il Cliente** deve poter consultare la scheda da mobile, utilizzare timer interattivi e inviare feedback.
- **Sicurezza:** L'accesso deve essere protetto da autenticazione JWT (JSON Web Token) con distinzione di ruoli.
- **Produzione:** Il sistema finale deve essere deployato su server Linux con Nginx.

2. Stack Tecnologico Richiesto

Backend (API & Data)

- **Linguaggio:** Python 3.x
- **Framework:** Flask (API RESTful)
- **Database:** SQLite (Relazionale)
- **Auth:** PyJWT (Gestione Token)
- **Server Prod:** Gunicorn (WSGI)

Frontend (UI & Logic)

- **Framework:** React.js (Vite)
- **Stile:** Tailwind CSS (Design System)
- **Routing:** React Router DOM (Gestione rotte protette)
- **Icons:** Lucide React

Infrastruttura & Deploy

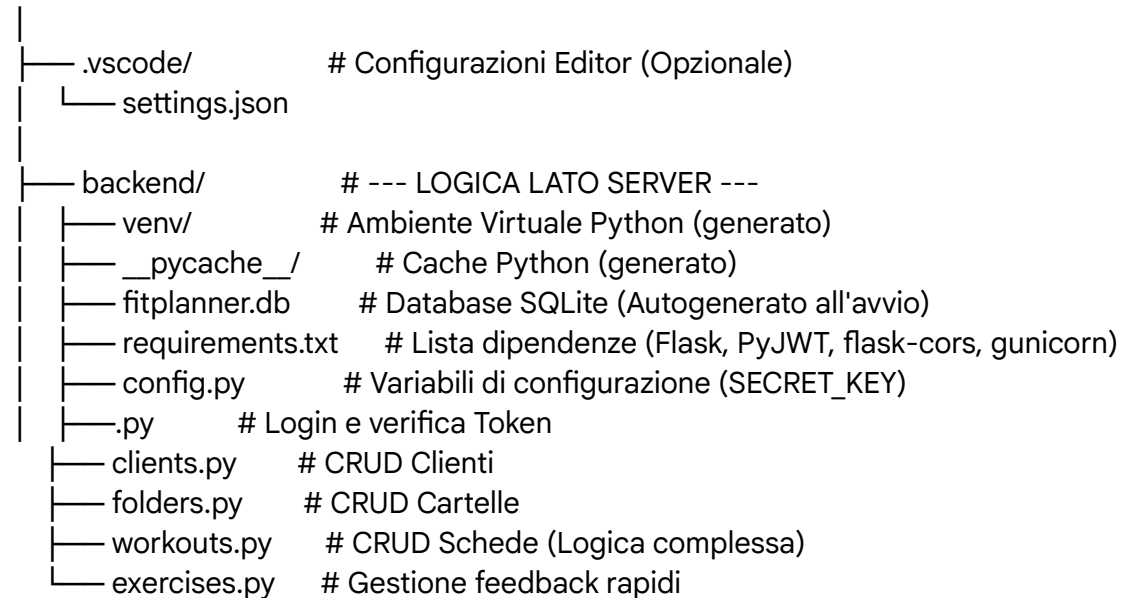
- **Server:** VPS Cloud (Linux Ubuntu/Debian)
- **Web Server:** NGINX (Reverse Proxy)

- **Build:** Node.js (npm run build)

3. Architettura Completa del File System

Questa struttura è tassativa. Ogni file e cartella ha uno scopo preciso.

FitPlanner/



Requisiti Funzionali Backend

1. Database (db.py):

- Tabelle necessarie: users (o clients/trainers distinti), folders, workouts, exercises.
- Relazioni: ON DELETE CASCADE per mantenere pulito il DB (es: se elimino una scheda, elimino i suoi esercizi).
- **Schema Esercizi:** Deve supportare la "Doppia Nota": una colonna trainer_notes (istruzioni) e una client_notes (feedback).

2. Autenticazione (auth.py):

- Implementare endpoint /api/auth/login.
- Riceve username e password.
- Verifica credenziali (hash SHA256 o bcrypt).
- Restituisce un **JWT Token** con scadenza (es. 24h) e il ruolo utente (trainer o client).
- Creare un decoratore @token_required per proteggere le rotte sensibili.

3. Logica Schede (workouts.py):

- **Create (POST):** Salva scheda e dettagli esercizi.
- **Update (PUT):** Logica "Tabula Rasa": aggiorna i dati di testa, cancella i vecchi esercizi associati e inserisce i nuovi (per gestire variazioni di serie/esercizi).
- **Read (GET):** Restituisce dati formattati per il frontend.

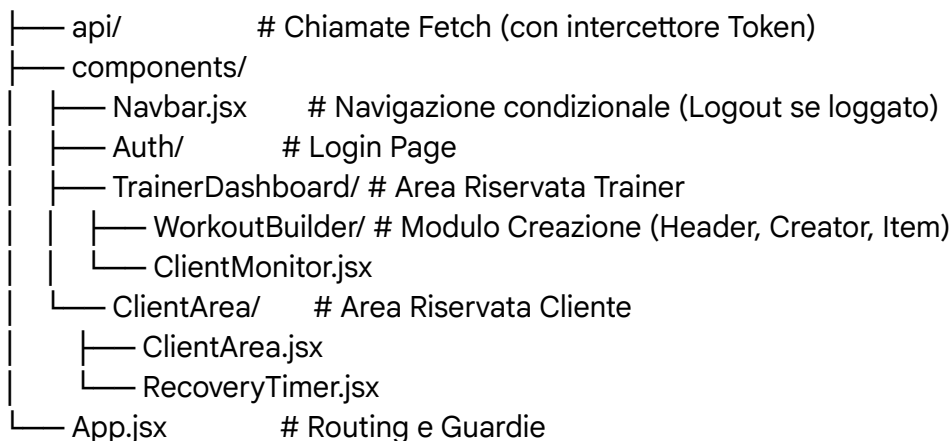
4. Fase 2: Sviluppo Frontend (L'Interfaccia)

Configurazione e Dipendenze

- Scaricare dipendenze: npm install
- Librerie chiave: react-router-dom, jwt-decode.
- **Environment Variables:** Creare file .env per gestire gli URL API dinamici.
 - .env.development: VITE_API_URL=http://127.0.0.1:5000/api
 - .env.production: VITE_API_URL=https://www.Lorenzoanzivino.it/api

Struttura Cartelle Frontend

frontend/src/



Requisiti Funzionali Frontend

1. **Sistema di Login:**
 - Pagina di login unificata.
 - Al submit, chiama API /login.
 - Salva il JWT in localStorage.
 - Reindirizza alla dashboard corretta in base al ruolo.
2. **Guarded Routes (Sicurezza):**
 - Creare componenti <ProtectedRoute> che avvolgono le rotte interne.
 - Se non c'è token o è scaduto -> Redirect a Login.
 - Se il ruolo è errato (es. Cliente cerca di andare su Dashboard Trainer) -> Redirect o Errore.
3. **Trainer Dashboard (Desktop & Tablet):**
 - **Builder Avanzato:** Griglia dinamica per inserire esercizi. Design "Dark Mode" (Grigio Scuro/Arancione).
 - **Monitor:** Visualizzazione "Collapsible" (a fisarmonica) delle schede dei clienti per risparmiare spazio.
4. **Client Area (Mobile First):**

- Design ottimizzato per smartphone.
- **Timer Recupero:** Il tempo di recupero deve essere un bottone cliccabile che avvia un countdown visivo.
- **Feedback:** Input di testo che salva automaticamente all'evento onBlur.

5. Fase 3: Deployment e Pubblicazione

Questa fase trasforma il codice in un sito web raggiungibile da chiunque.

Passo 1: Build del Frontend

- Eseguire comando: `npm run build`.
- Genera la cartella `dist/`. Questa cartella contiene HTML, CSS e JS statici pronti per la produzione.

Passo 2: Configurazione Server (Cloud Computing/VPS)

- Accesso al server remoto (Ionos) via SSH.
- Installazione requisiti: Python, Pip, Nginx.

Passo 3: Setup Backend Remoto

- Trasferire la cartella backend sul server.
- Configurare **Gunicorn** come server WSGI per servire l'applicazione Flask in modo performante (non usare `python app.py`).
- Creare un servizio systemd per far girare il backend in background 24/7.

Passo 4: Setup Frontend e NGINX

- Trasferire la cartella `dist` (generata al Passo 1) sul server (es: `/var/www/fitplanner`).
- Configurare **NGINX** come Reverse Proxy:
 - **Regola 1 (Frontend):** Tutte le richieste a `www.Lorenzoanzivino.it` devono servire i file statici della cartella `dist`.
 - **Regola 2 (API):** Tutte le richieste a `/api/*` devono essere inoltrate internamente a Gunicorn (es: `localhost:8000`).
 - **Regola 3 (SPA):** Gestire il fallback `try_files $uri /index.html` per far funzionare React Router al refresh della pagina.

6. Design System (Linee Guida Estetiche)

Tutta l'applicazione deve rispettare rigorosamente il seguente schema colori (Tailwind CSS):

- **Tema:** Dark Mode "Professional".
- **Sfondo App:** `bg-slate-100` (Login) / `bg-slate-900` (App interna).
- **Contenitori/Card:** `bg-slate-800` con bordi `border-slate-700`.
- **Input:** `bg-slate-900 text-white`.
- **Colore Primario (Azioni):** `orange-500` (Hover: `orange-600`).
- **Colore Feedback Positivo:** `emerald-500`